

Start coding or [generate](#) with AI.

```
import kagglehub
import pandas as pd

# Download latest version
path = kagglehub.dataset_download("uciml/default-of-credit-card-clients-dataset")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'default-of-credit-card-clients-dataset' dataset.
Path to dataset files: /kaggle/input/default-of-credit-card-clients-dataset

```
import os

# List the contents of the downloaded directory to find the correct filename
directory_contents = os.listdir(path)
print("Contents of the downloaded directory:", directory_contents)

# Assuming the correct file name is one of the listed files, or a commonly used one if present.
# For example, let's assume it's 'UCI_Credit_Card.csv' based on common Kaggle practices or by insp
# If the output shows a different name, this line will need to be adjusted.
# Let's try to find a common CSV file if multiple exist or assume a likely one.

# Let's find the first CSV file in the directory_contents
csv_files = [f for f in directory_contents if f.endswith('.csv')]

if len(csv_files) > 0:
    # Use the first CSV file found
    correct_csv_filename = csv_files[0]
    csv_file_path = os.path.join(path, correct_csv_filename)
    print(f"Attempting to load: {csv_file_path}")
    # Load the dataset into a pandas DataFrame
    df = pd.read_csv(csv_file_path)

    # Display the first 5 rows of the DataFrame
    display(df.head())
else:
    print("No CSV files found in the downloaded directory.")
    print("Please manually inspect the directory contents and specify the correct file.")
```

Contents of the downloaded directory: ['UCI_Credit_Card.csv']

Attempting to load: /kaggle/input/default-of-credit-card-clients-dataset/UCI_Credit_Card.csv

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_A
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	345
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	1494
3	4	50000.0	2	2	1	37	0	0	0	0	...	28314.0	2895
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	20940.0	1914

5 rows × 25 columns

```

from sklearn.metrics import roc_auc_score, roc_curve

# Calculate ROC-AUC for each model

# Decision Tree
y_pred_proba_dt = dt_classifier.predict_proba(X_test)[: , 1]
roc_auc_dt = roc_auc_score(y_test, y_pred_proba_dt)

# AdaBoost
y_pred_proba_ada = ada_classifier.predict_proba(X_test)[: , 1]
roc_auc_ada = roc_auc_score(y_test, y_pred_proba_ada)

# Random Forest
y_pred_proba_rf = rf_classifier.predict_proba(X_test)[: , 1]
roc_auc_rf = roc_auc_score(y_test, y_pred_proba_rf)

# Best Tuned Random Forest
y_pred_proba_best_rf = best_rf_model.predict_proba(X_test)[: , 1]
roc_auc_best_rf = roc_auc_score(y_test, y_pred_proba_best_rf)

print(f"ROC-AUC for Decision Tree: {roc_auc_dt:.4f}")
print(f"ROC-AUC for AdaBoost: {roc_auc_ada:.4f}")
print(f"ROC-AUC for Random Forest: {roc_auc_rf:.4f}")
print(f"ROC-AUC for Best Tuned Random Forest: {roc_auc_best_rf:.4f}")

```

```

ROC-AUC for Decision Tree: 0.6090
ROC-AUC for AdaBoost: 0.7704
ROC-AUC for Random Forest: 0.7570
ROC-AUC for Best Tuned Random Forest: 0.7737

```

✓ Model Comparison

```

import pandas as pd
from sklearn.metrics import precision_score, recall_score

# Extracting precision and recall for class 1 (default) from classification reports
# Function to parse report
def parse_report(report_str):
    lines = report_str.split('\n')
    metrics = {}
    for line in lines[2:-5]: # Skip header and averages
        if not line.strip():
            continue
        parts = line.split()
        if parts[0] == '1': # Metrics for class 1
            metrics['precision'] = float(parts[1])
            metrics['recall'] = float(parts[2])
            break
    return metrics

# Decision Tree metrics
dt_metrics = parse_report(report)
dt_precision = dt_metrics.get('precision')
dt_recall = dt_metrics.get('recall')

# AdaBoost metrics
ada_metrics = parse_report(report_ada)
ada_precision = ada_metrics.get('precision')
ada_recall = ada_metrics.get('recall')

```

```

# Random Forest metrics
rf_metrics = parse_report(report_rf)
rf_precision = rf_metrics.get('precision')
rf_recall = rf_metrics.get('recall')

# Best Tuned Random Forest metrics
best_rf_metrics = parse_report(report_best_rf)
best_rf_precision = best_rf_metrics.get('precision')
best_rf_recall = best_rf_metrics.get('recall')

# Create a DataFrame for comparison
comparison_df = pd.DataFrame({
    'Model': ['Decision Tree', 'AdaBoost', 'Random Forest', 'Tuned Random Forest'],
    'Accuracy': [accuracy, accuracy_ada, accuracy_rf, accuracy_best_rf],
    'Precision (Class 1)': [dt_precision, ada_precision, rf_precision, best_rf_precision],
    'Recall (Class 1)': [dt_recall, ada_recall, rf_recall, best_rf_recall],
    'ROC-AUC': [roc_auc_dt, roc_auc_ada, roc_auc_rf, roc_auc_best_rf]
})

# Display the comparison table
display(comparison_df.round(4))

```

	Model	Accuracy	Precision (Class 1)	Recall (Class 1)	ROC-AUC	
0	Decision Tree	0.7232	0.38	0.40	0.6090	
1	AdaBoost	0.8160	0.67	0.33	0.7704	
2	Random Forest	0.8119	0.63	0.36	0.7570	
3	Tuned Random Forest	0.8166	0.66	0.35	0.7737	

✓ Hyperparameter Tuning for Random Forest Classifier using GridSearchCV

```

from sklearn.model_selection import GridSearchCV

# Define the parameter grid for RandomForestClassifier
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20]
}

# Initialize GridSearchCV
grid_search_rf = GridSearchCV(estimator=RandomForestClassifier(random_state=42, n_jobs=-1), # n_j
                              param_grid=param_grid_rf,
                              cv=3, # 3-fold cross-validation
                              scoring='accuracy', # Evaluate based on accuracy
                              n_jobs=-1, # Use all available cores for parallel processing
                              verbose=1)

print("Starting GridSearchCV for Random Forest...")

# Fit GridSearchCV to the training data
grid_search_rf.fit(X_train, y_train)

print("GridSearchCV for Random Forest completed.")

```

```
# Get the best parameters and best score
best_params_rf = grid_search_rf.best_params_
best_score_rf = grid_search_rf.best_score_

print(f"\nBest Parameters for Random Forest: {best_params_rf}")
print(f"Best Cross-validation Accuracy for Random Forest: {best_score_rf:.4f}")

# Get the best model
best_rf_model = grid_search_rf.best_estimator_

# Make predictions with the best model
y_pred_best_rf = best_rf_model.predict(X_test)

# Evaluate the best model
accuracy_best_rf = accuracy_score(y_test, y_pred_best_rf)
report_best_rf = classification_report(y_test, y_pred_best_rf)
conf_matrix_best_rf = confusion_matrix(y_test, y_pred_best_rf)

print(f"\nBest Random Forest Model Accuracy on Test Set: {accuracy_best_rf:.4f}")
print("\nBest Random Forest Model Classification Report:")
print(report_best_rf)

print("\nBest Random Forest Model Confusion Matrix:")
print(conf_matrix_best_rf)

# Visualize the Confusion Matrix for the best Random Forest model
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_best_rf, annot=True, fmt='d', cmap='Oranges', cbar=False,
            xticklabels=['Not Default', 'Default'], yticklabels=['Not Default', 'Default'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix for Best Tuned Random Forest Classifier')
plt.show()
```

Starting GridSearchCV for Random Forest...
 Fitting 3 folds for each of 9 candidates, totalling 27 fits
 GridSearchCV for Random Forest completed.

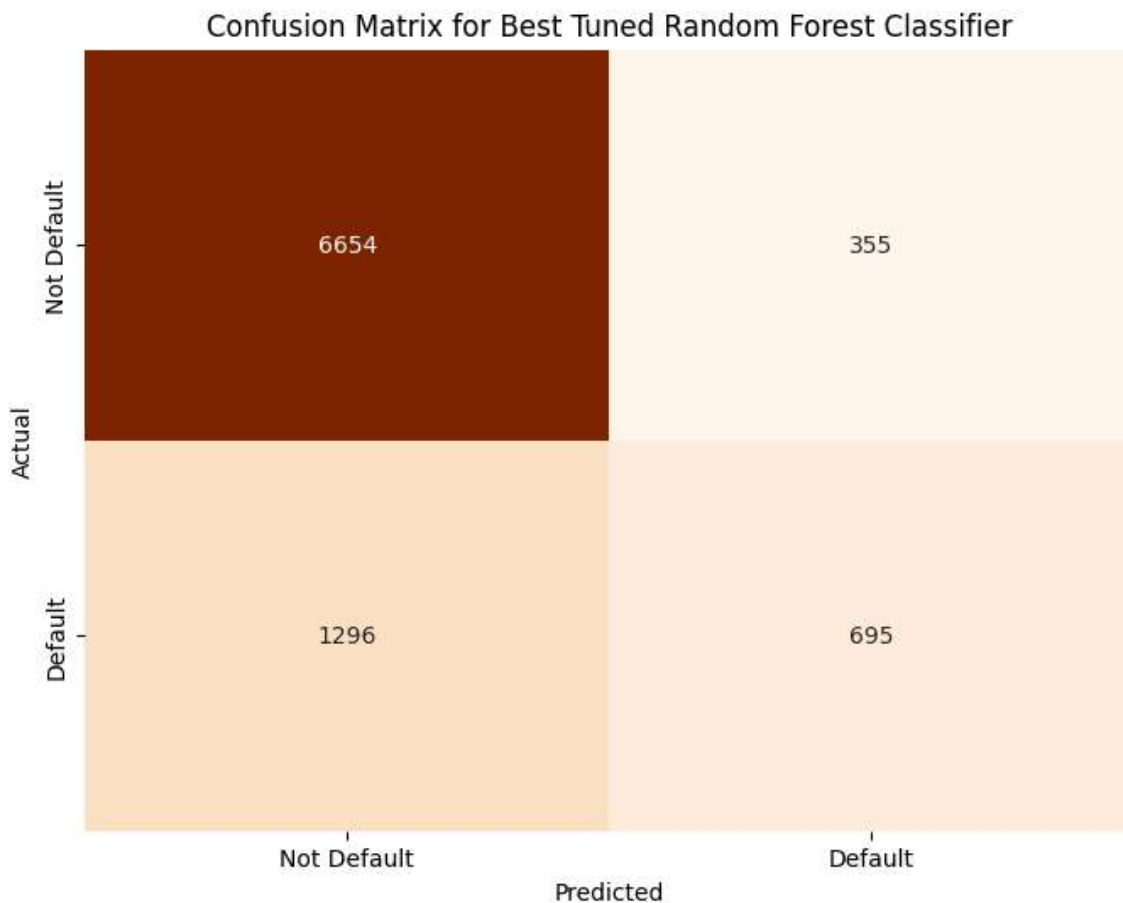
Best Parameters for Random Forest: {'max_depth': 10, 'n_estimators': 50}
 Best Cross-validation Accuracy for Random Forest: 0.8220

Best Random Forest Model Accuracy on Test Set: 0.8166

Best Random Forest Model Classification Report:

	precision	recall	f1-score	support
0	0.84	0.95	0.89	7009
1	0.66	0.35	0.46	1991
accuracy			0.82	9000
macro avg	0.75	0.65	0.67	9000
weighted avg	0.80	0.82	0.79	9000

Best Random Forest Model Confusion Matrix:
 [[6654 355]
 [1296 695]]



✓ Train and Evaluate AdaBoost Classifier

```
from sklearn.ensemble import AdaBoostClassifier

# Initialize the AdaBoost Classifier with a base estimator (e.g., Decision Tree)
ada_classifier = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=100)

# Train the model
```

```
ada_classifier.fit(X_train, y_train)

print("AdaBoost Classifier trained successfully.")

# Make predictions on the test set
y_pred_ada = ada_classifier.predict(X_test)

# Evaluate the model
accuracy_ada = accuracy_score(y_test, y_pred_ada)
report_ada = classification_report(y_test, y_pred_ada)
conf_matrix_ada = confusion_matrix(y_test, y_pred_ada)

print(f"\nAdaBoost Accuracy: {accuracy_ada:.4f}")
print("\nAdaBoost Classification Report:")
print(report_ada)

print("\nAdaBoost Confusion Matrix:")
print(conf_matrix_ada)

# Visualize the Confusion Matrix for AdaBoost
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_ada, annot=True, fmt='d', cmap='Greens', cbar=False,
            xticklabels=['Not Default', 'Default'], yticklabels=['Not Default', 'Default'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix for AdaBoost Classifier')
plt.show()
```

AdaBoost Classifier trained successfully.

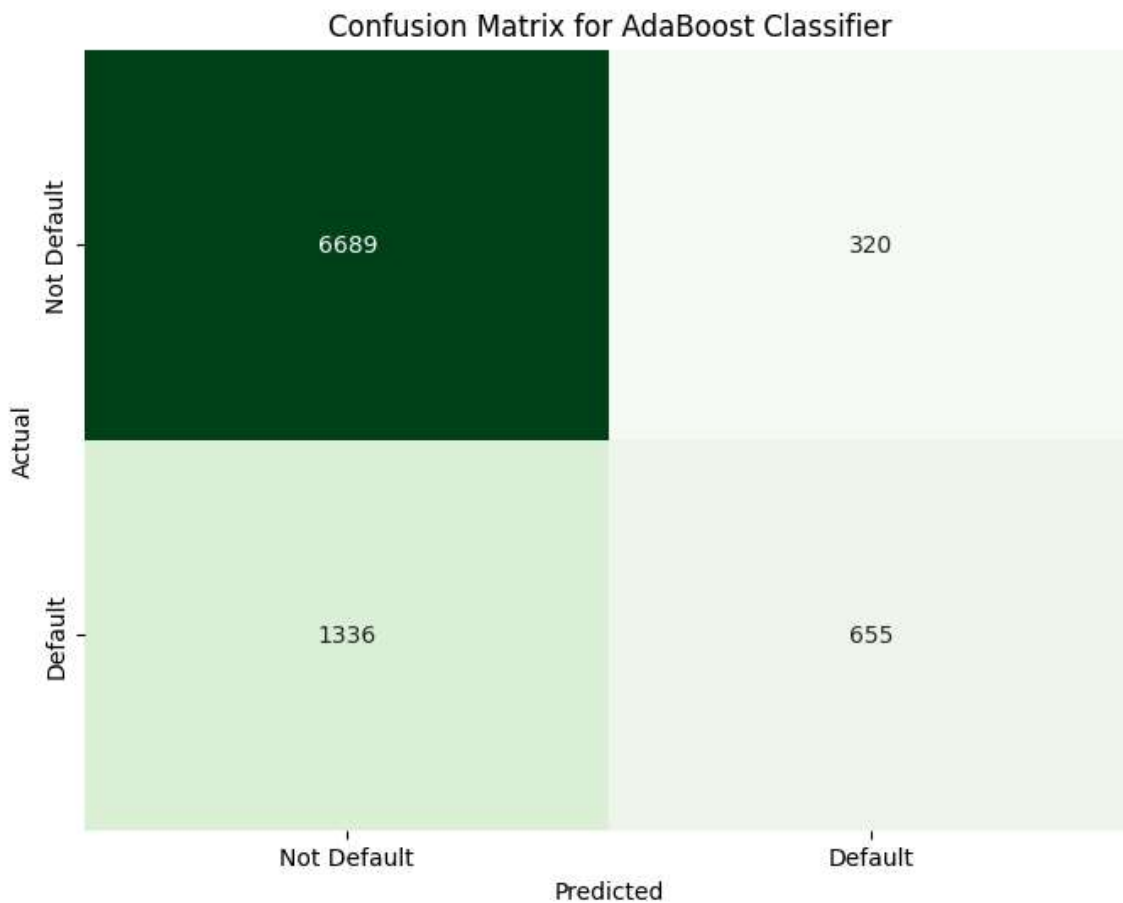
AdaBoost Accuracy: 0.8160

AdaBoost Classification Report:

	precision	recall	f1-score	support
0	0.83	0.95	0.89	7009
1	0.67	0.33	0.44	1991
accuracy			0.82	9000
macro avg	0.75	0.64	0.67	9000
weighted avg	0.80	0.82	0.79	9000

AdaBoost Confusion Matrix:

```
[[6689  320]
 [1336  655]]
```



✓ Train and Evaluate Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

# Initialize the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1) # n_jobs=-1

# Train the model
rf_classifier.fit(X_train, y_train)

print("Random Forest Classifier trained successfully.")
```

```
# Make predictions on the test set
y_pred_rf = rf_classifier.predict(X_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
report_rf = classification_report(y_test, y_pred_rf)
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)

print(f"\nRandom Forest Accuracy: {accuracy_rf:.4f}")
print("\nRandom Forest Classification Report:")
print(report_rf)

print("\nRandom Forest Confusion Matrix:")
print(conf_matrix_rf)

# Visualize the Confusion Matrix for Random Forest
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Purples', cbar=False,
            xticklabels=['Not Default', 'Default'], yticklabels=['Not Default', 'Default'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix for Random Forest Classifier')
plt.show()
```


Random Forest Classifier trained successfully.

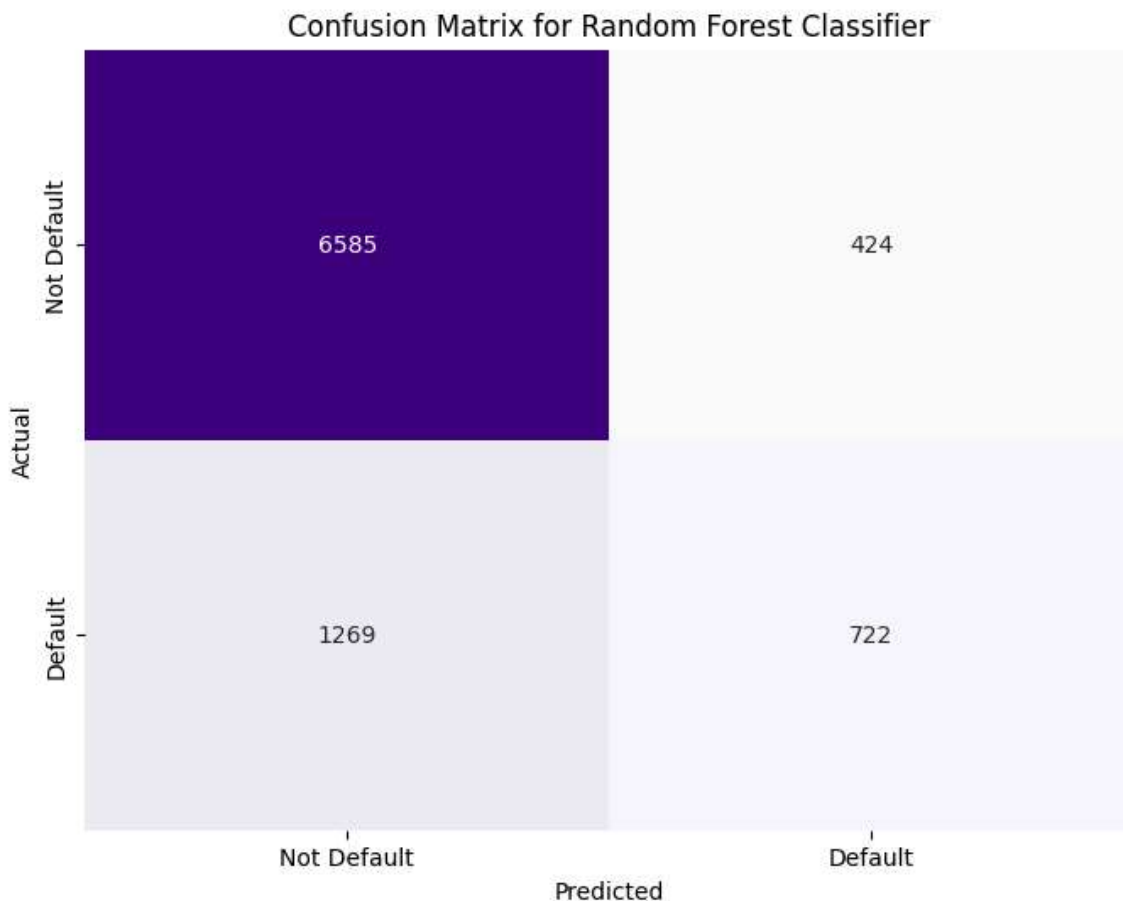
Random Forest Accuracy: 0.8119

Random Forest Classification Report:

	precision	recall	f1-score	support
0	0.84	0.94	0.89	7009
1	0.63	0.36	0.46	1991
accuracy			0.81	9000
macro avg	0.73	0.65	0.67	9000
weighted avg	0.79	0.81	0.79	9000

Random Forest Confusion Matrix:

```
[[6585  424]
 [1269  722]]
```



```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Define features (X) and target (y)
# The 'ID' column is typically not used as a feature for training.
X = df.drop(columns=['ID', 'default.payment.next.month'])
y = df['default.payment.next.month']
```

```
print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)
```

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, strat:

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)

Features (X) shape: (30000, 23)
Target (y) shape: (30000,)
X_train shape: (21000, 23)
X_test shape: (9000, 23)
y_train shape: (21000,)
y_test shape: (9000,)
```

Now that the data is split, let's train a Decision Tree Classifier.

```
# Initialize the Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state=42)
```